A Basic Instruction Logic for Electronic Computers

University of California

Los Alamos Scientific Laboratory

Bengt Carlson

February 14, 1956 T-1-101

The following is offered as a first attempt towards the development of a basic instruction logic for the new high-speed digital computer proposed for the Laboratory. It is hoped that this attempt, with improvements and details added, or subsequent attempts, will finally result in an instruction logic superior to what computers now in operation offer.

A digital computer is essentially a device capable of handling sequences of information. These may be sequences of instructions known as programs, or sequences of data, representing functions of a particularly simple type. The computer is capable of keeping sequences in storage, accepting such information from input devices, and generating new sequences by algebraic manipulation of one or several already in storage.

Formulae and Variables

New sequences of data are generated, as indicated above, by providing the machine with algebraic formulae in the form of programs. But is a program in present-day computers really a formula? A formula is supposed to have a fair amount of generality. A formula can be recognized and represents the same general mathematical operation no matter where it is recorded. Furthermore, once derived, it has the quality of permanence. A formula does not specify particular values of the independent variables, nor the ranges of these. When such information is given or used, we are applying or evaluating the formula.

It is, therefore, clear that <u>if a program is to represent a formula in the above sense, the instructions must not refer to particular sequences, nor to their ranges</u>, but rather to so-called dummy variables. Furthermore, <u>a stored program must be invariant with respect to relocation</u> in the memory. Anyone familiar with present-day machines will recognize that programs written for these are not formulae.

Note that in machine calculation the actual variables are referred to through parametric ones, in this case sequences of addresses.

Generalized Index Registers

An instruction logic will be proposed below capable of producing programs possessing the characteristics of formulae. Instructions will, as indicated, be constructed so as to refer to dummy variables. Operations such as "assembly" and "reassembly", terms familiar to 701 and 704 users, will become unnecessary, for programs will be made relocatable within the memory in the sense that data now are. Certain "set instructions" will be provided, enabling a program, when necessary, to refer to particular variables. The scheme as a whole does, by construction, eliminate the need for "automatic programming", in the sense that this term is now used. One may still desire mathematical language at the input stage, but the translation of this into instructions would essentially be a one-to-one proposition.

To accomplish the general goal the instructions will refer to index registers of a generalized type, capable of containing variables, and of being particularized by Set instructions. The latter would supply index registers with (1) a base address A, (2) a delta quantity D, and (3) a counting quantity N. The address variable is then given by $A_i \equiv A + N_i D$, $N_i = 0,1,2,\ldots, N-1$. Note that the instruction counter is a register of this type (with N missing), a fact which will be made good use of.

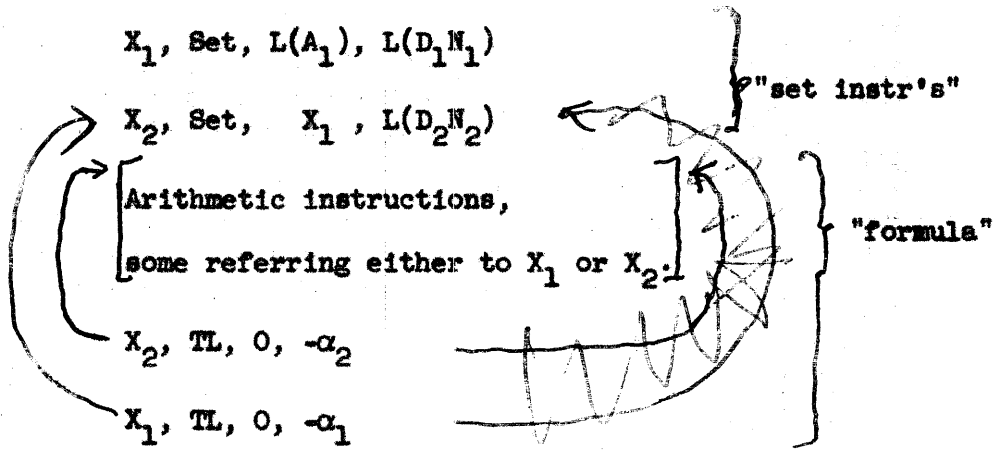
The instructions carry, in other words, the "address of an address". This type of instruction appeared in the IBM proposal to Livermore in the spring of 1955, and was again proposed by them at a recent SHARE meeting. Livermore expressed little or no interest in the idea, SHARE would examine it. Generalized, the concept appears to be fundamental. What significance IBM attaches to it, I do not know.

Arithmetic Instructions

Ordinary arithmetic instructions (AI) including transfer instructions associated with arithmetic are visualized as having two addresses and one "blank" to be discussed later. The "contents of" and the "address of" will be denoted by C( ) and L( ), respectively. For the moment, we write the AI instructions as follows:

blank, AI, X, $\pm\alpha$,

where α is an integer, and X the label of an index register, X = 0 representing, say, the instruction counter. Sequential information can be extracted without reference to the α-part. The operation then applies to $C[C_A(X)]$, $C_A(X)$ denoting the A-content of X. The extraction of absolute constants is provided for by giving a relative location $\pm\alpha$ with respect to $C_A(X)$. If X = 0 we get a location relative to the contents of the instruction counter. Transfer instructions for the purpose of controlling or repeating a piece of arithmetic are similarly given relative addresses. A multiple loop looks essentially as follows (the double loop case):



The Set instructions are thought of in these terms: The first one above, for instance, applies to index register X_1 (on the left), calls in a basic address A_1 and a number pair (D_1, N_1) from two separate locations, storing A_1, D_1, and N_1 in X_1. The second "Set" may refer to the index

register set by the first, as above. This makes it possible to "index indices" in chains of arbitrary length. One type of "Set" will, in addition, be called on to clear the accumulator (Acc), thus essentially eliminating the need for reset instructions of the type "Radd" (701).

The "loop transfer" (TL), for instance the last one above, refers to index register X_1. It causes D_1 to be added to the A_i in X_1, N_1 to be decreased by one, and, if N_1 becomes zero, to ignore the transfer. If $N_1 \neq 0$ the control passes to the second instruction.

Instructions corresponding to TSX and TXG in the 704 language are readily constructed to fit this general scheme. The TSX-type is important for linkages between programs. It stores the instruction counter in a specified index register and transfers to the specified location. The program now "remembers" the location it left and can return to the same or the general vicinity. The TXG-type is important in connection with boundary conditions at the extremes or within the range of a variable. It provides a means for testing N_1 to see if it has arrived at a particular value (boundary).

The Extraction of Information

The above details provide for the extraction of random data and the transfer to arbitrary locations, i.e., for the extraction of random information. It also provides for the reference to sequential information in multi-dimensional arrangements, i.e., data having addresses of the type $A_{i,j,k,\ldots}$. A third category consists of what one may call adjoining information. In numerical work this seems to be a separate and important class. In recursion formulae, for instance, one calls not only for A_i, but for each i also, say, for A_{i+1} and A_{i+2}. The "1" is really some D, the "2" twice that D. As another example consider a two-dimensional difference scheme involving $A_{i,j}$, $A_{i,j-1}$, $A_{i-1,j}$, and $A_{i-1,j-1}$. Here two D's are involved. To keep an index for each quantity is wasteful and

complicated. Besides, one wants to keep Set instructions to a minimum, preferably one per basic variable. If D is a fixed number, it may be entered as $\pm\alpha$ (see above), but this is not the most general case.

The following is a solution to the above problem though perhaps not as general as one might want it. One makes reference to an arbitrary D in storage by allowing for placing $\pm L(D)$ in the α-part of the instruction (the sign belonging to D). The instruction would now modify the address in X, replacing A_1 by $A_1 \pm D$ before performing the arithmetic operation. This requires an extension of the definition of X. We must now provide index registers with two A_1 locations. The Set instructions would set both and set them equal, and the TL instruction would equalize the two before proceeding with the operations assigned to it above. This provides for repeated modification of A_1's within a loop, hence for the extraction of data in a neighborhood, though not quite at random.

Two-Address Arithmetic

On examining, for instance, 701 programs, one finds that the arithmetic is often interrupted by the necessity for storing results in temporary locations. In the SNG code, to take an example, 2/3 of the Store instructions refer to temporary locations.

To remedy this a number of fast registers would be quite useful say, in addition to the Acc, another register (Acc #2) connected to the adder and several registers acting as fast storage. The "blank" could, for instance, be used as a tag to address these registers, say, even integers ("2" for Acc #2) to bring results to these registers, odd integers ("1" for Acc #2) to call on them for information. The inner product of two vectors would, as an example, then be formed by this program:

X_1, Set, $L(A_1)$, $L(D_1N_1)$
- , Add, X_1 , -
2 , Mpy, X_1 , $L(D_2)$
X_1, TL, 0 , -2
1 , Store, somewhere.

The first instruction sets up an index register for the multiplicand and clears the accumulators, the second brings the multiplicand to the Acc, the third modifies A_1 in X_1 by D_2, the distance, say, between the first addresses of the two vectors, whereupon it performs the multiplication, adding the result to Acc #2. By repetition the inner product is formed in Acc #2 from where it can be stored in the main memory if this is desired. The 701 requires some 17 instructions, the 704 twelve, for this simple operation.

Concluding Remarks

A computer with the above logical system becomes much more than a wizard at arithmetic; it has the makings of a remarkable research assistant. The machine comes to us with a blank memory. Eventually we make it memorize a very large number of formulae which it has ample place for in its auxiliary memory. To give a machine a problem eventually amounts to furnishing it with the data special to the problem and a simple program consisting mainly of Set instructions and basic linkages. The machine takes over, follows the instructions, remembers all of the formulae required, and comes up with the correct solution. Occasionally, one would have to give it a refresher course.

If we are to keep machines of the future, perhaps a hundred times faster than present ones, supplied with new programs we must make the latter efficiently refer to the accumulated knowledge, i.e., to the programs already written and in the simplest manner possible. This will, I think, assure success.

variable is given by an address

$$Var = A + iD \qquad i = 0, \cdots N-1$$

Set Variable: set A, set D, set N

"formula", $C_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$ $\quad i = 1 \cdots m$, $j = 1 \cdots m$

$k = 1 \cdots n$